

Requirement Analysis

Implement Client Script and UI Policy (Incident)

	Name	NM.Id
Team Leader	Yamuna devi A	asaluau020224128037
Team Members	Kalaiselvi S	asaluau020224128018
	Keerthana M	asaluau020224128020
	Kavitha V	asaluau020224128019
	Mageshwari R	asaluau020224128021

1. Project Title

Implement Client Script and UI Policy for Incident Management in Service Now

2. Introduction

Service Now Incident Management is used to record, track, manage, and resolve incidents reported by users. The Incident form contains various fields that are required to capture complete information about an issue. In some situations, the behaviour of these fields needs to change based on the user's input. For example, a field may need to become mandatory, read-only, or visible only when a particular condition is satisfied. To achieve this dynamic behaviour, Client Scripts and UI Policies can be implemented on the Incident form. These features help improve data accuracy, reduce unnecessary user input, and provide a better user experience.

3. Problem Statement

The Incident form may contain several fields that are not always required for every situation. If all fields behave in the same way regardless of the user's selections, users may enter incomplete or unnecessary information. The following problems may occur:

- * Important fields may be left empty.
- * Users may modify fields that should not be changed.
- * Unnecessary fields may remain visible.
- * Incorrect information may be submitted.
- * Additional validation may be required before saving an Incident.

Therefore, a dynamic form configuration is required to control the behaviour of Incident fields according to predefined conditions.

4. Objectives

The main objective of this project is to implement Client Script and UI Policy functionality on the Incident form. The specific objectives are:

1. To provide dynamic behaviour for Incident fields.
2. To improve data validation.
3. To make required fields mandatory when necessary.
4. To control field visibility based on conditions.
5. To make selected fields read-only when required.
6. To reduce incorrect and incomplete Incident records.
7. To improve the overall user experience while creating and updating incidents.

5. Functional Requirements

5.1 Incident Form

The configuration should be implemented on the Incident table. The Incident form should allow users to create and update incident records while applying the required dynamic field behaviour. The system should ensure that the required information is entered before the Incident is submitted.

5.2 Client Script

A Client Script should be created to perform client-side actions based on the specified requirement. The Client Script should execute when the defined form event occurs. Depending on the requirement, it may be executed during:

- * Form loading
- * Field value change
- * Form submission

The script should perform the required action without unnecessarily affecting other fields on the Incident form.

5.3 UI Policy

A UI Policy should be created to dynamically control field properties. Based on the specified condition, a UI Policy can control whether a field is:

- * Mandatory
- * Visible
- * Read-only

When the condition is satisfied, the required UI Policy Action should be applied automatically.

6. Field Behaviour Requirements

Mandatory Fields

When a particular condition is satisfied, the required field should become mandatory. The user should provide the required information before submitting the Incident.

Field Visibility

Fields that are relevant only under certain conditions should be displayed when required. Unnecessary fields can remain hidden to keep the form simple.

Read-Only Field

Certain fields may need to be protected from modification. When the defined condition is satisfied, the field should become read-only while still allowing the user to view its value. These dynamic field behaviours help maintain consistency and reduce incorrect data entry.

7. User Experience Requirements

The Incident form should be simple and easy to use. Users should be able to understand which information is required and which fields can be modified. Changes made by the Client Script or UI Policy should be reflected immediately without unnecessary page refreshes. The system should provide clear validation or error messages whenever required information is missing or incorrect. The configuration should reduce confusion and help users complete the Incident form correctly.

8. Validation Requirements

The system should validate the Incident information before the record is submitted. The validation should ensure that:

- * Required fields contain information.
- * User input follows the defined conditions.
- * Mandatory fields are completed.
- * Incorrect or incomplete information is identified.
- * Valid Incident records can be submitted successfully.

Client-side validation should help improve the accuracy and completeness of Incident records.

9. Testing Requirements

After implementing the Client Script and UI Policy, the configuration should be tested using different Incident scenarios. The following tests should be performed:

1. Open an Incident form and verify the initial field behaviour.
2. Change the field related to the condition and verify the dynamic behaviour.
3. Check whether the required field becomes mandatory.
4. Verify that the specified field becomes visible or hidden correctly.
5. Verify that the required field becomes read-only when applicable.

6. Change the condition back and verify the expected normal behaviour.
7. Submit a valid Incident and confirm that it is saved successfully.
8. Attempt to submit incomplete information and verify that appropriate validation is applied.

Testing ensures that the Client Script and UI Policy work correctly without affecting other Incident functionality.

10. Expected Outcome

After successful implementation, the Incident form should behave dynamically according to the defined requirements. The expected outcome is:

- * Client Script executes correctly.
- * UI Policy condition works correctly.
- * Required fields become mandatory when necessary.
- * Fields are displayed or hidden according to the condition.
- * Restricted fields become read-only when required.
- * Incorrect or incomplete information is reduced.
- * Users receive appropriate guidance while entering Incident information.
- * Incident records can be created and updated successfully.

11. Benefits

The implementation provides several benefits to Incident Management. It improves data quality by ensuring that important information is captured. Dynamic field behaviour also reduces unnecessary data entry and helps users focus only on the information relevant to the current Incident. Client Scripts provide flexible client-side behaviour, while UI Policies provide an easier way to control field properties without requiring complex scripting for every requirement. Overall, the configuration improves the consistency, accuracy, and usability of the Incident form.

12. Conclusion

The implementation of Client Script and UI Policy on the Incident form provides an effective way to control field behaviour and improve data validation in Service Now. Client Scripts can be used to respond to user actions and perform client-side operations, while UI Policies can dynamically control field properties such as mandatory, visible, and read-only states. By implementing and testing these requirements properly, the Incident form can provide a more organized and user-friendly experience while maintaining accurate and complete Incident records.